

Section A: Conceptual Understanding

1. Explain why Express is preferred over plain Node HTTP servers for REST APIs.

Express is favored over simple Node.js HTTP servers when designing RESTful APIs as it offers an ease of development and maintainability benefit.

Using the basic HTTP server in Node.js, developers will be responsible for defining their routing rules, request parsing, logic processing, error handling, and generating appropriate responses. Such a development pattern will most likely increase the amount of coding effort required.

What Express offers:

Routing: Defining REST endpoints through `app.get()`, `app.post()`, `app.put()`, and `app.delete()` functions.

Middlewares: Easily implementing features such as authentication, logging, validation, and error handling.

Request & Response Processing Tools: Having access to features such as `req.params`, `req.body`, and `res.json()`.

Code Modularization: Making development easier through modularizing routes and middleware.

Community: Having numerous third-party plugins that can work well with Express.

2. Explain designing RESTful routes for products and categories.

Routing in REST is resource oriented and uses resources like products and categories. HTTP methods are used to perform actions on those resources.

Products Endpoints:

- GET /products: Get all products
- GET /products/:id: Get particular product
- POST /products: Add a product
- PUT /products/:id: Update a product
- DELETE /products/:id: Delete a product

Categories Endpoints:

- GET /categories: Get all categories
- GET /categories/:id: Get a category
- POST /categories: Add a category
- PUT /categories/:id: Update a category
- DELETE /categories/:id: Delete a category

3. Explain middleware usage for request validation.

Request data validation middleware is used to check if the incoming data in a request is valid before it gets to the route handler.

Advantages of using such middleware:

- Ensures that invalid data does not get into your system.
- Helps to keep your route handlers free since they will not perform any validations.
- Boosts security and integrity of the data.

4. Explain JWT authentication flow for protected APIs.

JWT authentication technique is utilized for securing APIs and limiting access to secure routes only for authenticated users.

Steps:

1. User provides his username or email along with a password for authentication.
2. Server verifies these details provided by the user.
3. After verifying these details successfully, a JWT is issued by the server to the user.
4. This token is stored by the user in his local storage or cookie.
5. On accessing protected APIs, user sends this token in Authorization header.
6. Server verifies the token; if it is valid, then access is allowed, else not allowed.

5. Explain why API contracts are critical before frontend integration.

API contracts are very crucial before integrating the frontend and backend because they set out the ways of communicating between systems. In particular, they state the endpoints of APIs, request/response structure, and error handling mechanisms.

Advantages:

- Allowing frontend and backend teams to work independently.
- Fewer misunderstandings or challenges during integration.
- Offering clear request and response format guidelines.
- Easier to test and debug.

For instance, where the API contract states:

```
{ "id": 1, "name": "Laptop", "price": 50000 }
```

it will enable the frontend team to know exactly what data to expect and how to display it.

In conclusion the API contract is used to facilitate interaction between the frontend and backend teams.

Section B: Practical Tasks

1. Category & Product CRUD: Develop an Express.js server with distinct route files for /categories and /products. Implement standard HTTP methods—GET, POST, PUT, and DELETE—to manage the catalog structure.

```
////server.js

const express = require('express');

const app = express();

app.use(express.json());

app.use('/categories', require('./routes/categories'));
app.use('/products', require('./routes/products'));

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

2. MongoDB Integration: Establish a connection using Mongoose to define schemas for both Categories and Products. Use ref and populate() to create a relational link between a product and its specific category.

```
//routes//categories.js
```

```
const express = require('express');
```

```
const router = express.Router();
```

```
let categories = [  
  { id: 1, name: 'Electronics' },  
  { id: 2, name: 'Clothing' }  
];
```

```
// GET all categories
```

```
router.get('/', (req, res) => {  
  res.json(categories);  
});
```

```
// GET category by ID
```

```
router.get('/:id', (req, res) => {  
  const category = categories.find(  
    c => c.id === parseInt(req.params.id)  
  );  
  
  if (!category) {  
    return res.status(404).json({ message: 'Category not found' });  
  }  
});
```

```
    res.json(category);
  });

// POST create category
router.post('/', (req, res) => {
  const newCategory = {
    id: categories.length + 1,
    name: req.body.name
  };

  categories.push(newCategory);

  res.status(201).json(newCategory);
});

// PUT update category
router.put('/:id', (req, res) => {
  const category = categories.find(
    c => c.id === parseInt(req.params.id)
  );

  if (!category) {
    return res.status(404).json({ message: 'Category not found' });
  }

  category.name = req.body.name;
```

```

    res.json(category);
  });

// DELETE category
router.delete('/:id', (req, res) => {
  categories = categories.filter(
    c => c.id !== parseInt(req.params.id)
  );

  res.json({ message: 'Category deleted' });
});

module.exports = router;

```

3. JWT Admin Authentication: Protect sensitive routes (like adding or deleting products) using JSON Web Tokens (JWT). Create a middleware to verify the token in the request header and check for "admin" privileges before granting access.

```

const express = require('express');
const router = express.Router();

let products = [
  {
    id: 1,
    name: 'Laptop',

```

```
    price: 50000,  
    categoryId: 1  
  },  
  {  
    id: 2,  
    name: 'T-Shirt',  
    price: 1000,  
    categoryId: 2  
  }  
];
```

```
// GET all products
```

```
router.get('/', (req, res) => {  
  res.json(products);  
});
```

```
// GET product by ID
```

```
router.get('/:id', (req, res) => {  
  const product = products.find(  
    p => p.id === parseInt(req.params.id)  
  );  
  
  if (!product) {  
    return res.status(404).json({ message: 'Product not found' });  
  }  
  
  res.json(product);  
});
```



```
});
```

```
// POST create product
```

```
router.post('/', (req, res) => {  
  const newProduct = {  
    id: products.length + 1,  
    name: req.body.name,  
    price: req.body.price,  
    categoryId: req.body.categoryId  
  };  

```

```
  products.push(newProduct);
```

```
  res.status(201).json(newProduct);  
});
```

```
// PUT update product
```

```
router.put('/:id', (req, res) => {  
  const product = products.find(  
    p => p.id === parseInt(req.params.id)  
  );
```

```
  if (!product) {  
    return res.status(404).json({ message: 'Product not found' });  
  }
```

```
  product.name = req.body.name;
```

```
product.price = req.body.price;  
product.categoryId = req.body.categoryId;
```

```
res.json(product);  
});
```

```
// DELETE product
```

```
router.delete('/:id', (req, res) => {  
  products = products.filter(  
    p => p.id !== parseInt(req.params.id)  
  );
```

```
  res.json({ message: 'Product deleted' });  
});
```

```
module.exports = router;
```

3. Multer Image Upload: Integrate the Multer middleware to handle multipart/form-data. Configure storage settings to save product images to a local /uploads folder and store the resulting file path in the MongoDB document.

test Endpoints

Categories

- GET /categories
- POST /categories
- PUT /categories/1
- DELETE /categories/1

Products

- GET /products
- POST /products
- PUT /products/1
- DELETE /products/1

This fulfills the requirement of distinct route files for /categories and /products with GET, POST, PUT, and DELETE CRUD operations.

Product Management API Objective: Build a scalable E-Commerce backend using Node.js and Express.js with advanced image handling and secure authentication.

ecommerce-api/

```
|
|
|— config/
|   |— db.js
|
|
|— models/
|   |— Category.js
|   |— Product.js
|
|
|— routes/
|   |— categoryRoutes.js
|   |— productRoutes.js
|   |— authRoutes.js
|
|
|— middleware/
|   |— authMiddleware.js
|   |— uploadMiddleware.js
|
|
|— uploads/
|
|
|— server.js
|— .env
|— package.json
```

- 1. Product & Category CRUD Implement full Create, Read, Update, and Delete operations for both products and categories. Use Mongoose to establish a relationship between them (e.g., linking a Product to its Category via ObjectId) to ensure a structured and searchable catalog.**

```
const mongoose = require("mongoose");
```

```
const categorySchema = new mongoose.Schema({  
  name: {  
    type: String,  
    required: true  
  }  
});
```

```
module.exports = mongoose.model("Category", categorySchema);
```

2. Image Upload with Validation Integrate Multer middleware to handle multipart/form-data for product images. Implement strict validation to filter file types (e.g., only JPEG/PNG) and limit file sizes, ensuring the server remains optimized and secure from malicious uploads.

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  name: String,

  price: Number,

  image: String,

  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Category"
  }
});
```

```
module.exports = mongoose.model("Product", productSchema);
```

3.JWT-Protected Routes Secure administrative endpoints using JSON Web Tokens (JWT). Create a custom middleware that verifies the token in the Authorization header, allowing only logged-in admins to perform sensitive tasks like adding new products or deleting categories.

```
const multer = require("multer");
```

```
const storage = multer.diskStorage({
  destination: "./uploads",

  filename: (req, file, cb) => {
    cb(null, Date.now() + "-" + file.originalname);
  }
});
```

```
const fileFilter = (req, file, cb) => {
  const allowedTypes = [
    "image/jpeg",
    "image/png"
  ];
  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error("Only JPG and PNG allowed"));
  }
};
```

```
module.exports = multer({
  storage,
  limits: {
    fileSize: 2 * 1024 * 1024
  },
  fileFilter
});
```

```
const jwt = require("jsonwebtoken");

module.exports = (req, res, next) => {

  const token =
    req.header("Authorization")?.replace("Bearer ", "");

  if (!token) {
    return res.status(401).json({
      message: "Access Denied"
    });
  }

  try {

    const verified = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    req.user = verified;

    next();

  } catch (error) {

    res.status(403).json({
```



```
message: "Invalid Token"
```

```
});
```

```
}
```

```
};
```